sumo_morl

Többcélú megerősítéses tanulás jelzőlámpa-vezérléshez

Kézikönyv

Kivonat

A `sumo_morl` egy Python könyvtár, amellyel többcélú megerősítéses tanulással (MORL) lehet jelzőlámpa-vezérlő ágenst fejleszteni a SUMO forgalomszimulátorban, forgalomtechnikai előismeretek nélkül. A csomópont, a jelzésterv, a biztonsági (közbenső) idők, a detektorok és a forgalom készen van; a feladat kizárólag az ágens megírása. Ez a kézikönyv elmagyarázza, mit modellez a könyvtár, mit lát és mit csinálhat az ágens, mi alapján kap jutalmat, és hogyan érdemes dolgozni vele. A programkód belső felépítését a *Programfelépítés* dokumentum, az egyes függvények és osztályok részletes leírását az *API-referencia* (`docs/sumo_morl/api/index.html`) tartalmazza.

Tartalomjegyzék

1. Bevezetés **2**

1.1. Mi a feladat? . . . 2

1.2. Mit ad a könyvtár, és mit kell megírni? . . . 2

1.3. Alapfogalmak röviden . . . 2

2. Telepítés és első futtatás **3**

2.1. Telepítés . . . 3

2.2. Demó: klasszikus fix idejű program . . . 3

2.3. A legkisebb saját program . . . 3

3. A szimulált csomópont **4**

3.1. Geometria . . . 4

3.2. Jelzőcsoportok, konfliktusok és fázisok . . . 4

3.3. Közbenső idők . . . 5

3.4. Fázisátmenetek . . . 6

4. Hogyan vezérel az ágens? **6**

4.1. Akciók . . . 6

4.2. Változó lépéshossz . . . 7

4.3. Epizódok és forgalom . . . 7

5. Mit lát az ágens? **7**

5.1. Csak mérhető adatok . . . 7

5.2. A detektorok 8
5.3. Becsült járműszám (bemenet–kimenet módszer) 8
5.4. A megfigyelésvektor 8
5.5. Amit szándékosan nem lát az ágens 8

6. Mit tanul az ágens? – célfüggvények **9**
6.1. Vektor-jutalom 9
6.2. Saját célfüggvény 9
6.3. Pareto-front és skalarizálás 9

7. Munka a könyvtárral **10**
7.1. Ajánlott munkafolyamat 10
7.2. Összehasonlító vezérlők 10
7.3. Kiértékelés kódban 11
7.4. Tanítás Stable-Baselines3-mal 11

8. Feladatötletek **11**

A. Gyakori problémák **12**

B. A példaprogramok **13**

1. Bevezetés

1.1. Mi a feladat?

Egy jelzőlámpás csomópontban a vezérlő folyamatosan arról dönt, melyik irány kapjon zöldet. Egy jó vezérlés kevés várakozással, kevés megállással és igazságosan szolgálja ki a járműveket. Ezek a célok gyakran ellentmondanak egymásnak:

- ha mindig a legtöbb járművet kiszolgáló irány kap zöldet, a kis forgalmú mellékág sokat vár;
- ha gyakran váltunk, mindenki hamar sorra kerül, de minden váltás néhány másodperc „elveszett időt" jelent (sárga, piros), ami csökkenti a csomópont áteresztőképességét.

Ezért nincs egyetlen legjobb vezérlés: csak *kompromisszumok* vannak. A többcélú megerősítéses tanulás (*multi-objective reinforcement learning*, MORL) erre ad eszközöket: a jutalom nem egy szám, hanem egy **vektor**, célonként egy elemmel, és a cél nem egyetlen ágens, hanem a lehetséges kompromisszumok (a Pareto-front) feltérképezése.

1.2. Mit ad a könyvtár, és mit kell megírni?

Készen van	**A hallgató feladata**
T-csomópont hálózata a SUMO-ban	az ágens (pl. DQN, PPO, saját MORL algoritmus)
jelzőcsoportok, fázisok, biztonságos fázisátmenetek	a célfüggvények kiválasztása, esetleg saját cél
indukcióshurok-detektorok, mérések aggregálása	a tanítás és a kiértékelés megtervezése
véletlen, reális forgalmi igény	az eredmények (Pareto-front) elemzése
Gymnasium környezet vektor-jutalommal	
összehasonlító (klasszikus) vezérlők, MORL segédfüggvények, ábrák	

Az ágens minden döntési pontban **egyetlen dolgot** mond meg: melyik fázis legyen a következő. A környezet gondoskodik róla, hogy a jelzőlámpák ezután szabályosan és biztonságosan váltsanak.

1.3. Alapfogalmak röviden

Ágens, környezet, lépés
Az ágens megfigyelést kap, akciót választ, a környezet szimulál egy darabig, majd új megfigyelést és jutalmat ad vissza. Egy *epizód* egy szimulált időszak (alapértelmezés szerint egy óra).

Vektor-jutalom
Minden lépésben egy k elemű vektor $\mathbf{r}_t = (r_t^1, \ldots, r_t^k)$, ahol k a célok száma. Az epizód vektor-hozama $\mathbf{R} = \sum_t \mathbf{r}_t$.

Dominancia, Pareto-front
Egy $\mathbf{R}_a$ hozam *dominálja* $\mathbf{R}_b$-t, ha minden célban legalább olyan jó, és legalább egyben jobb. A nem dominált hozamok halmaza a *Pareto-front*.

Skalarizálás
A vektor-jutalom egy számmá alakítása, pl. súlyozott összeggel: $r_t = \mathbf{w}^\top \mathbf{r}_t$, ahol $\mathbf{w} \geq 0$, $\sum_i w_i = 1$. Különböző súlyokkal a front különböző pontjai érhetők el.

2. Telepítés és első futtatás

2.1. Telepítés

Python 3.9 vagy újabb kell. A csomag gyökérkönyvtárában (ahol a `pyproject.toml` van):

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -e .                 # a könyvtár és a SUMO
pip install -e ".[rl]"           # opcionális: Stable-Baselines3 a tanítási példához
python -m pytest                 # ellenőrzés: a tesztek kb. fél perc alatt lefutnak
```

A SUMO-t az `eclipse-sumo` pip csomag hozza magával, külön telepíteni nem kell. Ha a gépen már van SUMO, és a `SUMO_HOME` környezeti változó be van állítva, a könyvtár azt használja.

2.2. Demó: klasszikus fix idejű program

Mielőtt bármilyen ágenst írnál, nézd meg a működést a legegyszerűbb vezérléssel. A fix idejű program a fázisokat mindig ugyanabban a sorrendben, mindig ugyanannyi ideig tartja zölden (alapértelmezés: 30, 10 és 15 másodperc), a forgalomtól függetlenül.

```
python examples/sumo_morl/00_demo_fix_ido.py --gui              # SUMO grafikus felü
    lettel
python examples/sumo_morl/00_demo_fix_ido.py --green 40 10 20   # más zöldidőkkel
```

A program először kiírja a fázisokat, a köztük lévő átmeneteket és a ciklusidőt (a ciklus egy teljes kör az összes fázison; itt $30 + 4 + 10 + 5 + 15 + 5 = 69$ s). Ezután minden fázisváltásnál kiírja a detektorokból becsült járműszámot ágankként, a végén pedig az epizód jutalmait és néhány valós mutatót (átlagos késés, utazási idő). Elmenti az első két ciklus jelzéstervét is (1. ábra).



1. ábra. A fix idejű program első két ciklusa jelzőcsoportonként. Zöld: szabad jelzés; sárga: a zöld vége; narancs: piros-sárga (indulás előtt); piros: tilos. A keleti egyenes sáv két egymást követő fázisban is zöld, ezért a zöldje hosszabb.

Érdemes kipróbálni

Futtasd a demót `-gui` kapcsolóval, és figyeld meg a SUMO felületén a sárga és a piros-sárga jelzéseket, a balra kanyarodók külön sávját, és azt, hogyan nő a sor a mellékágon, amíg piros. Utána próbálj ki más zöldidőket, és figyeld, hogyan változnak a jutalmak.

2.3. A legkisebb saját program

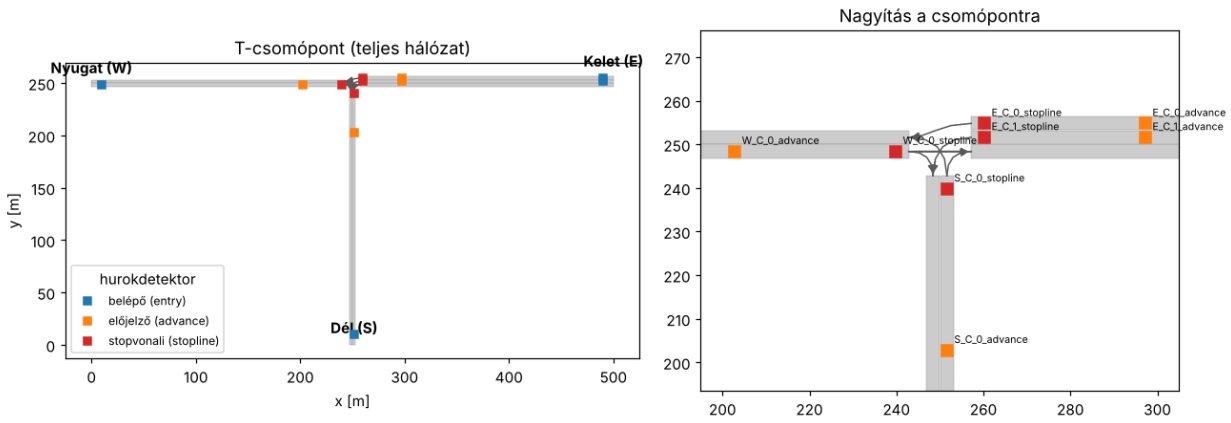
```python
import numpy as np
from sumo_morl import TJunctionMOEnv

env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"))
obs, info = env.reset(seed=0)
R = np.zeros(env.reward_dim)
done = False
while not done:
    action = env.action_space.sample()      # ide kerül az ágens döntése
    obs, reward, terminated, truncated, info = env.step(action)
    R += reward                             # a reward egy vektor
    done = terminated or truncated
print(dict(zip(env.objective_names, R)))
env.close()
```

3. A szimulált csomópont

3.1. Geometria

Egyszerű T-csomópont jobbkezes forgalommal (2. ábra). Mindhárom ág 250 m hosszú, a megengedett sebesség 50 km/h. A főút nyugat–kelet irányú, a mellékág délről csatlakozik.



2. ábra. A csomópont és a hurokdetektorok. Balra a teljes hálózat, jobbra nagyítás a csomópontra.

Ág	Bejövő sáv	Mozgások
Nyugat (W)	`W_C_0`	egyenesen keletre és jobbra délre (közös sáv)
Kelet (E)	`E_C_0`	egyenesen nyugatra
Kelet (E)	`E_C_1`	balra, délre (külön kanyarodósáv)
Dél (S)	`S_C_0`	balra nyugatra és jobbra keletre (közös sáv)

A keleti bejövő ág tehát két sávra van osztva: a balra kanyarodók külön sávban várnak, így nem tartják fel az egyenesen haladókat. Visszafordulás nincs.

3.2. Jelzőcsoportok, konfliktusok és fázisok

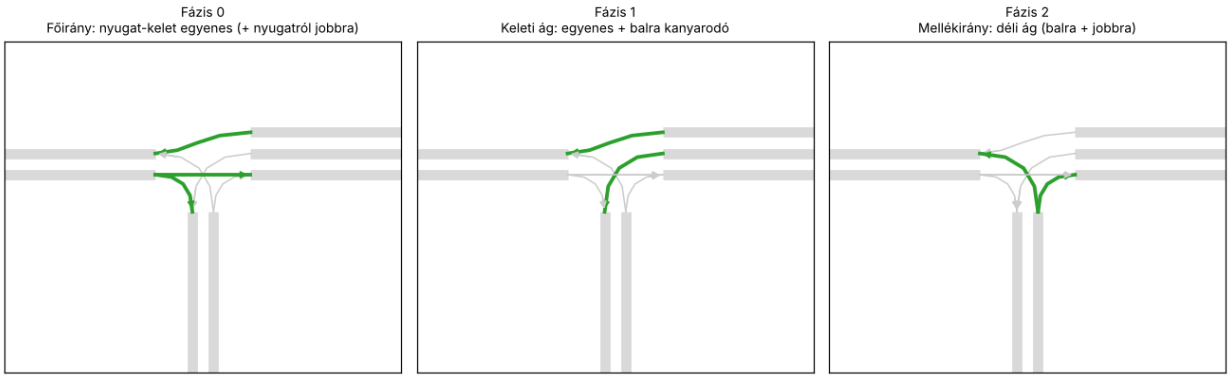
Jelzőcsoport
Mozgások halmaza, amelyek mindig ugyanazt a jelzést kapják. Itt egy bejövő sáv egy jelzőcsoport, így négy csoport van: `Ny_egyenes+jobb`, `K_egyenes`, `K_balra`, `D_bal+jobb`.

Konfliktus
Két csoport konfliktusos, ha járműveik útvonala keresztezi egymást vagy összefut. Konfliktusos csoportok nem kaphatnak egyszerre zöldet.

Fázis
Egymással nem konfliktusos csoportok halmaza, amelyek együtt kapnak zöldet.

A fázisokat a könyvtár a konfliktusokból számolja: a konfliktusmentes csoportok *maximális* halmazai lesznek a fázisok (gráfelméleti nyelven: a kompatibilitási gráf klikkjei). Ebből a csomópontból pontosan három fázis adódik (3. ábra):

Fázis	Zöldet kap	Leírás
0	`W_C_0`, `E_C_0`	főirány mindkét irányban egyenesen (és nyugatról jobbra)
1	`E_C_0`, `E_C_1`	keleti ág: egyenes és védett balra kanyarodás
2	`S_C_0`	déli mellékág



3. ábra. A három fázis. Zölddel a szabad jelzést kapó mozgások.

3.3. Közbenső idők

Amikor egy irány zöldje véget ér, az utolsó, még zölden behajtó jármű egy ideig a csomópontban van. A vele konfliktusos irány csak akkor kaphat zöldet, amikor ez a jármű már biztosan elhagyta az ütközési pontot. Az ehhez szükséges minimális időt nevezzük **közbenső időnek** (*intergreen*): $k(i \to j)$ az i csoport zöldjének vége és a j csoport zöldjének kezdete között legalább eltelő idő.

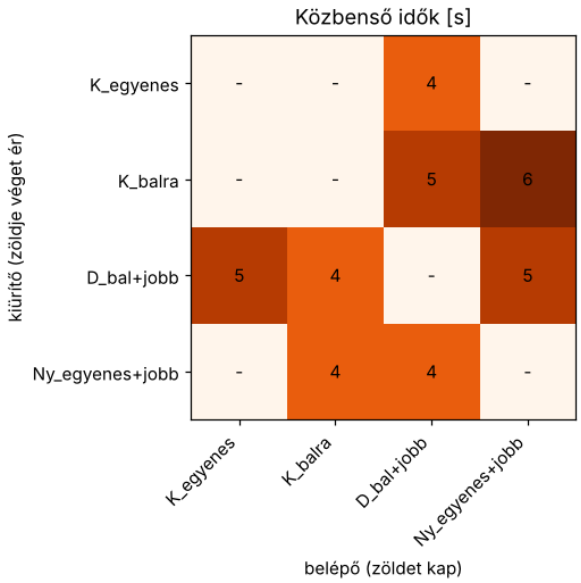
A közbenső időket a mellékelt közbensőidő-számító kód (`geometry_utils.py`, `sumo_parser.py`) számolja a csomópont geometriájából:

1. minden mozgás pályáját simított görbével közelíti, ebből kanyarodási sugarat, abból jellemző sebességet számol;
2. megkeresi a pályák ütközési pontjait;
3. minden ütközési pontra kiszámolja

$$k = t_{\text{át}} + \frac{s_{\text{ki}} + l_{\text{jármű}}}{v_{\text{ki}}} - \frac{s_{\text{be}}}{v_{\text{be}}},$$

ahol $t_{\text{át}} = 3$ s, $l_{\text{jármű}} = 6$ m, s_{ki} a kiürítő jármű útja a stopvonaltól az ütközési pontig, s_{be} a belépő jármű útja az ütközési pontig, v_{ki} és v_{be} a sebességeik. Az eredményt egész másodpercre felfelé kerekíti.

A jelzőcsoportok közötti közbenső idő a csoportok mozgásai közötti értékek maximuma (4. ábra).



4. ábra. Közbenső idők [s] a jelzőcsoportok között. Sor: a csoport, amelynek a zöldje véget ér; oszlop: a csoport, amely zöldet kap; „–”: nincs konfliktus.

3.4. Fázisátmenetek

Fázisváltáskor a környezet automatikusan lejátssza az átmenetet:

- a megszűnő csoportok 3 s **sárgát**, majd pirosat kapnak;
- az induló csoportok akkor kapnak zöldet, amikor a közbenső idők ezt megengedik (legalább 2 s után), előtte 2 s **piros-sárgát**;
- a mindkét fázisban zöld csoportok végig zöldek maradnak.

Az átmenetek hossza másodpercben:

honnan \ hova	0	1	2
0	–	4	4
1	6	–	5
2	5	5	–

Megjegyzés

Az átmenetek alatt egyik konfliktusos irány sem haladhat, ezért ez az idő a forgalom szempontjából elveszett. Ez az egyik oka annak, hogy a túl gyakori váltás rossz.

4. Hogyan vezérel az ágens?

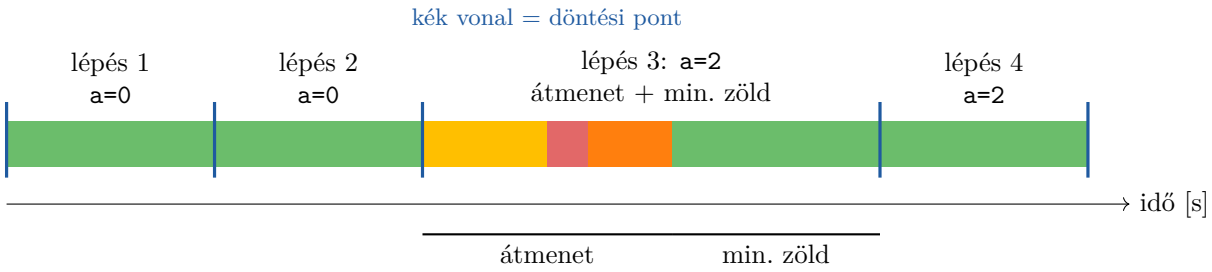
4.1. Akciók

Az akciótér `Discrete(3)`: az akció a **következő fázis** sorszáma.

- **Ha az akció az aktuális fázis**, a zöld `decision_interval` (alapértelmezés 5) másodperccel meghosszabbodik, utána jön a következő döntés.

- **Ha más fázist választ**, lefut a szabályos átmenet (4–6 s), majd az új fázis `min_green` (alapértelmezés 5) másodpercig biztosan zöld. Csak ezután jön a következő döntés.

Bármelyik akció mindig érvényes, szabálytalan jelzéskép nem állhat elő. Opcionálisan megadható egy legnagyobb zöldidő (`max_green`); ha egy zöld ennél tovább tartana, a környezet kikényszeríti a váltást a következő fázisra.



5. ábra. Döntési pontok az időben. Maradás esetén a lépés 5 s, váltáskor az átmenettel és a minimális zölddel együtt 9–11 s.

4.2. Változó lépéshossz

A 5. ábrán látható, hogy a lépések hossza nem állandó. Az adott lépés hossza másodpercben az `info["elapsed"]` értékben található. A jutalmak a lépés másodperceire vannak összegezve, így az epizód összjutalma a lépéshosszaktól függetlenül a teljes időre vett összeg. Ha diszkontálva tanulsz, érdemes lehet a diszkontot a lépés hosszához igazítani (γ^{elapsed}); ez jó haladó feladat.

4.3. Epizódok és forgalom

Egy epizód alapértelmezés szerint 3600 szimulált másodperc. Minden `reset()` új epizódot indít új, véletlen forgalommal: a főirányon irányonként 200–700, a mellékágon 80–400 jármű óránként, 10–45%-os kanyarodási arányokkal. A járművek véletlen időpontokban (Poisson-folyamat szerint) érkeznek. Azonos `seed` azonos forgalmat ad, így a vezérlők igazságosan összehasonlíthatók.

Rögzített igény is megadható:

```
from sumo_morl import TJunctionMOEnv, Demand

env = TJunctionMOEnv(demand=Demand.constant(main=600, side=250))
env = TJunctionMOEnv(demand=lambda rng: Demand.random(rng, main_range=(400, 800)))
env.reset(options={"demand": Demand.constant(300, 100)})   # csak erre az epizódra
```

5. Mit lát az ágens?

5.1. Csak mérhető adatok

Egy valódi csomópont vezérlője nem látja a járműveket: nem tudja, hány autó áll a sorban, ki mióta vár vagy hová tart. Csak azt tudja, amit az úttestbe épített **indukcióshurok-detektorok** mérnek. Ahhoz, hogy a tanult vezérlés a valóságba is átvihető legyen, a könyvtár megfigyelése és jutalma **kizárólag ilyen mérhető adatokból** készül.

5.2. A detektorok

Minden bejövő sávon (összesen 4) három hurok van, így összesen 12:

Típus	Helye	Mire jó
belépő (`entry`)	10 m-re az ág elejétől	a csomópont felé érkező járművek számlálása
előjelző (`advance`)	40 m-re a stopvonaltól	közeledő járművek; ha tartósan foglalt, a sor idáig ér
stopvonali (`stopline`)	3 m-re a stopvonaltól	az áthaladó járművek számlálása, a sor eleje

Egy hurok minden lépésben négy mennyiséget ad:

`count`	a lépés alatt áthaladt járművek száma
`occupancy`	foglaltság: az idő hányadában volt jármű a hurok felett (0–1)
`speed`	átlagsebesség a hurkon [m/s]; ha nem volt jármű, a megengedett sebesség
`gap`	a legutóbbi érzékelés óta eltelt idő [s] (időrés)

5.3. Becsült járműszám (bemenet–kimenet módszer)

A belépő és a stopvonali hurok számlálóiból folyamatosan számolható, hány jármű van a kettő között:

$$N(t) = N(t-1) + \text{belépők}(t) - \text{kilépők}(t).$$

Ezt ágankként számoljuk (a keleti ág két sávját együtt, mert a járművek sávot válthatnak). A tesztek szerint a becslés legfeljebb néhány járművel tér el a szimuláció valós értékétől.

5.4. A megfigyelésvektor

A megfigyelés egy 55 elemű, $[0, 1]$ tartományba normalizált vektor:

Tartalom	Elemszám	Normalizálás
detektoronként `count`, `occupancy`, `speed`, `gap`	$12 \times 4 = 48$	lásd lent
becsült járműszám ágankként (W, E, S)	3	az ág tárolókapacitásához (≈7 m/jármű)
aktuális fázis (one-hot)	3	0 vagy 1
az aktuális zöld eddigi hossza	1	/120 s

A detektoradatok normalizálása: a forgalom (jármű/óra) 1800 jármű/órához, a sebesség a sebességkorláthoz, az időrés 60 s-hoz viszonyítva, mindegyik 1-nél levágva. Az egyes elemek neve az `env.observation_names()` függvénnyel kérdezhető le (pl. `'E_C_1_stopline.occupancy'`).

A nyers, aggregált detektoradatok az `info["detectors"]` objektumban is elérhetők, így saját megfigyelés is építhető (pl. `gymnasium.ObservationWrapper`-rel).

5.5. Amit szándékosan nem lát az ágens

Jármű-azonosítók, pozíciók, úticélok, pontos sorhossz, várakozási idő, emisszió. Ezeket a környezet az `info["ground_truth"]` alatt kiírja, de **csak kiértékeléshez**.

Ne csalj!

Az `info["ground_truth"]` adatait (pl. átlagos késés, CO_2) használd a dolgozatban annak bemutatására, hogy a vezérlőd tényleg jobb. A megfigyelésbe vagy a jutalomba viszont ne építsd be őket, mert a valóságban nem mérhetők.

6. Mit tanul az ágens? – célfüggvények

6.1. Vektor-jutalom

A környezet létrehozásakor választod ki a célokat; a jutalomvektor ebben a sorrendben tartalmazza őket:

```
env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"))  # alapértelmezés
env.objective_names        # ('queue', 'fairness', 'switches')
```

Konvenció: **nagyobb érték = jobb**, a költségek negatív előjelűek. Minden cél a lépés másodperceire összegzett mennyiség, és egy rögzített skálával szorozva van, hogy a tipikus lépésenkénti értékek 1 körüliek legyenek.

Név	Mit mér (lépésenként)	Mit jelent
`queue`	$-\sum_{\text{ágak}} \sum_{\text{mp}}$ becsült járműszám, $\times 0{,}01$	a teljes késleltetés közelítése: minél kevesebb jármű álljon a csomópont előtt
`fairness`	$-\max_{\text{ág}} \sum_{\text{mp}}$ becsült járműszám, $\times 0{,}02$	méltányosság: a legrosszabb helyzetű ág se várjon sokat
`switches`	-1, ha volt fázisváltás	stabil vezérlés, kevés elveszett idő
`throughput`	stopvonalon áthaladt járművek, $\times 0{,}1$	áteresztőképesség (főleg túltelítettségben számít)
`stopline_occupancy`	$-\sum$ stopvonali foglalt másodpercek, $\times 0{,}1$	a stopvonalnál álló járművek közelítése

Mind csak detektoradatból és a vezérlő saját állapotából számol.

6.2. Saját célfüggvény

Egy célfüggvény egy `(snap, ctx) -> float` függvény: `snap` az adott lépés aggregált detektoradatai, `ctx` egy szótár (`phase_changed`, `phase`, `duration`, `time`, `forced`).

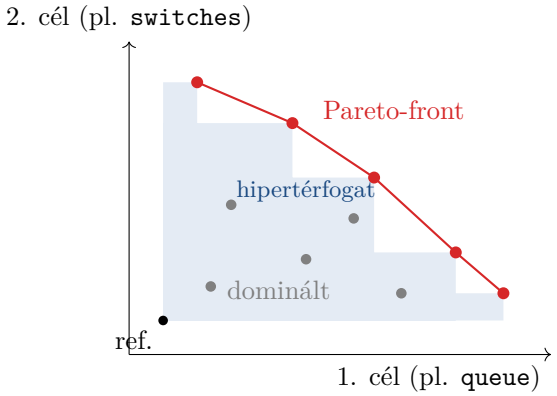
```
from sumo_morl import TJunctionMOEnv, register_objective

@register_objective("side_road_queue", scale=0.02, doc="A déli ág becsült járműszáma")
def side_road_queue(snap, ctx):
    return -snap.approach_vehicle_seconds["S"]

env = TJunctionMOEnv(objectives=("queue", "side_road_queue"))
```

6.3. Pareto-front és skalarizálás

A 6. ábra a fogalmakat szemlélteti két célra. Minden pont egy vezérlés átlagos vektor-hozama; a piros pontok nem domináltak, ezek alkotják a frontot.



6. ábra. Pareto-front két maximalizálandó célra. A kék terület a hipertérfogat a referenciaponthoz képest: minél nagyobb, annál jobb és szerteágazóbb a front.

A legtöbb RL algoritmus skalár jutalmat vár. Két egyszerű megoldás van készen:

- **Lineáris skalarizálás** (`LinearScalarization`): $r = \mathbf{w}^\top \mathbf{r}$ rögzített súllyal. Különböző súlyokkal külön ágenseket tanítva a front konvex része feltérképezhető.
- **Súly-kondicionált ágens** (`WeightConditioned`): minden epizódban véletlen súlyt sorsol, és azt a megfigyelés végére fűzi. Egyetlen háló tanulja meg az összes kompromisszumot; kiértékeléskor a kívánt súlyt kell megadni.

Mindkét wrapper az eredeti vektort is visszaadja az `info["vector_reward"]` alatt.

7. Munka a könyvtárral

7.1. Ajánlott munkafolyamat

1. **Ismerkedés**: demó, majd `01_ismerkedes.py`.
2. **Alapvonal**: a klasszikus vezérlők kiértékelése azonos forgalmon (`02_baseline_osszehasonlitas.py`).
3. **Tanítás**: egy vagy több ágens tanítása (pl. `03_sb3_sulyozott_tanitas.py`).
4. **Kiértékelés**: vektor-hozamok azonos forgalmon, Pareto-front, hipertérfogat, valós mutatók.

7.2. Összehasonlító vezérlők

`RandomPolicy`	véletlen fázis
`FixedTimePolicy`	fix idejű, ciklikus program
`ActuatedPolicy`	járműkövető: amíg az aktuális fázis előjelző hurkain rövid időközönként jön jármű, hosszabbít (legfeljebb `max_green`-ig), utána a következő olyan fázisra vált, ahol van igény
`LongestQueuePolicy`	mohó: a legnagyobb becsült igényű fázist választja

Egy tipikus eredmény (1800 s, három véletlen forgalom átlaga):

vezérlő	`queue`	`fairness`	`switches`	átl. álló idő [s]
véletlen	−463	−735	−148	59
fix 30/10/15	−175	−226	−78	17
járműkövető	−136	−157	−121	9
leghosszabb sor	−134	−156	−112	9

Jól látszik a kompromisszum: a fix idejű program kevesebbet vált, de többet várnak a járművek. Egy jó MORL ágensnek legalább a fix idejű programot kell vernie, és érdemes a járműkövetővel is összevetni.

7.3. Kiértékelés kódban

```
from sumo_morl import (TJunctionMOEnv, FixedTimePolicy, evaluate_policy,
                       pareto_mask, hypervolume, plot_pareto)

env = TJunctionMOEnv()
res = evaluate_policy(env, FixedTimePolicy(env, [30, 10, 15]), seeds=[0, 1, 2])
res["mean"], res["std"], res["ground_truth"]["mean_delay_per_vehicle"]
```

Egy Stable-Baselines3 modell az `sb3_policy(model)` segítségével értékelhető ki ugyanígy. A Pareto-front a `pareto_mask`, a hipertérfogat a `hypervolume`, az ábra a `plot_pareto` függvénnyel készíthető. Különböző módszereket mindig ugyanazokkal a `seed`-ekkel és ugyanazzal a referenciaponttal hasonlíts össze.

7.4. Tanítás Stable-Baselines3-mal

```
from stable_baselines3 import DQN
from sumo_morl import TJunctionMOEnv, LinearScalarization

env = LinearScalarization(TJunctionMOEnv(ground_truth=False), weights=[0.6, 0.3, 0.1])
model = DQN("MlpPolicy", env, verbose=1).learn(total_timesteps=50_000)
model.save("dqn_w06_03_01")
```

Egy egyórás epizód kb. 400–700 döntést jelent, és egy átlagos gépen néhány másodperc alatt fut le. Tanításhoz érdemes a `ground_truth=False` beállítás, mert gyorsabb. Több környezet párhuzamosan is futtatható (pl. `SubprocVecEnv`), mindegyik saját SUMO-folyamatot indít.

8. Feladatötletek

Bevezető

1. Futtasd a demót `-gui`-val. Melyik fázisváltás a leghosszabb, és miért?
2. Keress olyan fix idejű zöldidőket, amelyek a `queue` célban jobbak az alapértelmezettnél. Melyik másik cél romlik közben?

Egycélúból többcélúba

3. Taníts DQN-t vagy PPO-t 5–10 különböző súlyvektorral. Rajzold fel a Pareto-frontot, számold ki a hipertérfogatot, és vesd össze a klasszikus vezérlőkkel.

4. Taníts egyetlen súly-kondicionált ágenst, és értékeld ki ugyanazokkal a súlyokkal. Melyik front jobb, és mennyivel kevesebb tanítás kellett hozzá?
5. Definiálj saját célfüggvényt, és vizsgáld meg, hogyan változik a front.

Haladó

6. Valósíts meg egy vektor-Q-értékes MORL algoritmust (pl. Envelope Q-learning).
7. Használj nemlineáris (pl. Csebisev) skalarizálást. Találsz-e pontot a front nem konvex részén?
8. Írj wrappert, amely a diszkontot a lépés hosszához igazítja (γ^{elapsed}).
9. Tanítsd az ágenst alacsony forgalmon, és értékeld ki csúcsórás profillal (`Demand.random(rng, profile=(0.5, 1, 1.5, 1))`). Mennyire robusztus?
10. Hagyj el egy detektortípust a megfigyelésből. Melyik detektor a legfontosabb az ágensnek?

Kiértékelési tanácsok

- Mindig azonos forgalmon (azonos `seed`) hasonlíts össze vezérlőket.
- Legalább 3–5 epizód átlagát és szórását add meg.
- A jutalmakon kívül a valós mutatókat (`info["ground_truth"]`) is közöld.

A. Gyakori problémák

„Nem található a(z) 'sumo' program"
Telepítsd a csomagot `pip install -e .` paranccsal (ez hozza a SUMO-t), vagy állítsd be a `SUMO_HOME` változót a meglévő SUMO-telepítésre.

`ModuleNotFoundError: No module named 'sumo_morl' vagy 'traci'`
A csomag nincs telepítve az éppen használt Pythonba. A csomag gyökérkönyvtárában: `python3 -m venv .venv`, `source .venv/bin/activate`, `pip install -e .`, és a példákat ugyanebben a terminálban futtasd.

A `-gui` nem nyit ablakot
Távoli gépen vagy Colabban nincs grafikus felület; ott a `gui=False` használható. macOS-en a SUMO grafikus felületéhez szükség lehet az XQuartz telepítésére.

Lassú a tanítás
`ground_truth=False`, rövidebb epizód (`episode_length`), párhuzamos környezetek.

`RuntimeError: Előbb hívd meg a reset()-et!`
A `step()` előtt mindig kell egy `reset()`.

Egyszerre több környezet
Lehetséges; minden környezet saját SUMO-folyamatot indít. Használat után hívd meg az `env.close()`-t.

B. A példaprogramok

`00_demo_fix_ido.py`	klasszikus fix idejű program, eseménynapló, jelzésterv-ábra
`01_ismerkedes.py`	a jelzésterv és a detektorok kiírása, véletlen ágens
`02_baseline_osszehasonlitas.py`	klasszikus vezérlők azonos forgalmon, Pareto-ábra
`03_sb3_sulyozott_tanitas.py`	DQN több súlyvektorral, Pareto-front, hipertérfogat
`04_sajat_celfuggveny.py`	saját célfüggvények definiálása